

University of Tehran
Farabi College
Advanced Online Business Course

Project Report
Arbaeen Pilgrim Assistant Chatbot

Submitted by
Mohammed Razzaq Mohsen Al-Tayyar

Student ID

220704109

Table of Contents

1. Introduction.....	1
2. Artificial Intelligence Usage Report	1
2.1 Overview	1
2.2 AI Tools Used.....	1
2.2.1 Claude (Anthropic).....	2
2.2.2 ChatGPT (OpenAI)	2
2.2.3 Groq API with Llama 3.1 8B	2
2.3 How AI Output Was Evaluated	2
2.4 Boundaries of AI Use	2
3. System Architecture.....	3
3.1 Overview	3
3.2 Architecture Components	3
3.2.1 Frontend Layer	3
3.2.2 AI Layer	4
3.2.3 Deployment Layer.....	4
3.3 Data Flow	4
3.4 Technology Stack Summary.....	4
4. Velocity Chart.....	5
4.1 Definition.....	5
4.2 Velocity Data.....	5
4.3 Analysis	6
5. Lessons Learned.....	6
5.1 Project Management	6
5.2 AI-Assisted Development.....	6
5.3 System Prompt Engineering	6
5.4 Deployment and Environment Management.....	7
5.5 Scope Management.....	7
5.6 What I Would Do Differently.....	7
6. Conclusion	7
7. Project Links	8
7.1 Live Application	8
7.2 GitHub Repository.....	8

1. Introduction

The Arbaeen pilgrimage is one of the largest annual human gatherings on earth, drawing an estimated 20 to 25 million pilgrims to Karbala, Iraq each year. Pilgrims walk approximately 80 kilometers from Najaf to Karbala on foot, facing significant practical challenges along the way including locating Mawkib food and rest stations, managing health issues such as heat exhaustion and dehydration, and finding reliable route guidance in a crowded and dynamic environment.

This report documents the development of the Arbaeen Pilgrim Assistant Chatbot, a web-based conversational AI system built to address these challenges. The system allows pilgrims to ask questions in natural language and receive immediate, practical guidance across three core areas: Mawkib station locations, first aid advice, and route information. The project was developed as part of the Advanced Online Business course at the University of Tehran, Farabi College, following an agile methodology with four sprints over six weeks.

The report is organized into four sections. Section 1 documents the artificial intelligence tools used during development and how their outputs were evaluated. Section 2 describes the system architecture and the technical components of the solution. Section 3 presents the project velocity data across all sprints. Section 4 reflects on the key lessons learned throughout the development process.

2. Artificial Intelligence Usage Report

2.1 Overview

Artificial intelligence tools played a central and intentional role throughout the development of the Arbaeen Pilgrim Assistant Chatbot. Rather than treating AI as an optional aid, the project embraces AI assistance as a core part of the development workflow, which aligns directly with the philosophy stated in the course brief. This section documents which tools were used, for what purposes, and how their outputs were evaluated and refined before being incorporated into the final product.

2.2 AI Tools Used

2.2.1 Claude (Anthropic)

Claude served as the primary development assistant throughout the project. It assisted in writing and structuring all React components including the Chat interface and the Groq API service file. It also helped design the Arbadeen-specific system prompt that constrains the chatbot to relevant topics. Furthermore, Claude assisted in writing all project documentation including this report, the product backlog, the sprint board, and the burndown chart. In each case, the generated output was reviewed, adjusted for accuracy, and refined to match the project context before use.

2.2.2 ChatGPT (OpenAI)

ChatGPT was used as a secondary reference tool for cross-checking technical decisions, particularly around React state management and API integration patterns. It also assisted in verifying the accuracy of Arbadeen-related factual content that was embedded in the system prompt, such as the Iraqi emergency number 122 and general route information between Najaf and Karbala.

2.2.3 Groq API with Llama 3.1 8B

The Groq API with the Llama 3.1 8B Instant model serves as the core AI engine of the deployed product itself. This model processes all pilgrim queries and generates responses in real time. The model was selected based on its availability on the Groq free tier, its response speed, and its demonstrated ability to handle domain-specific conversational queries accurately during testing.

2.3 How AI Output Was Evaluated

Every AI-generated component passed through a review process before being used. For code, each function was tested manually in the browser to confirm correct behavior. For the system prompt, multiple test queries were submitted covering all three core use cases: Mawkib guidance, first aid advice, and route questions. Responses were evaluated for relevance, accuracy, and tone. For documentation, all AI-drafted text was read carefully and rewritten where it contained inaccurate claims, vague language, or a writing style inconsistent with the project requirements.

2.4 Boundaries of AI Use

All final decisions regarding architecture, feature scope, and content accuracy were made by the developer. AI tools provided suggestions and drafts, but they did not replace engineering judgment. The developer takes full responsibility for the quality and correctness of the final product. No AI-generated content was submitted without review and verification.

3. System Architecture

3.1 Overview

The Arbaeen Pilgrim Assistant Chatbot follows a lightweight client-side architecture with no dedicated backend server. All application logic runs in the browser, and communication with the AI model happens directly through the Groq REST API. This design was chosen deliberately to keep the system simple, maintainable, and deployable within the constraints of a course project with no infrastructure budget. Figure 1 illustrates the complete system architecture including the request and response flow between the user, the React frontend, the Groq API, and the Llama 3.1 model, as well as the deployment pipeline connecting GitHub and Vercel.

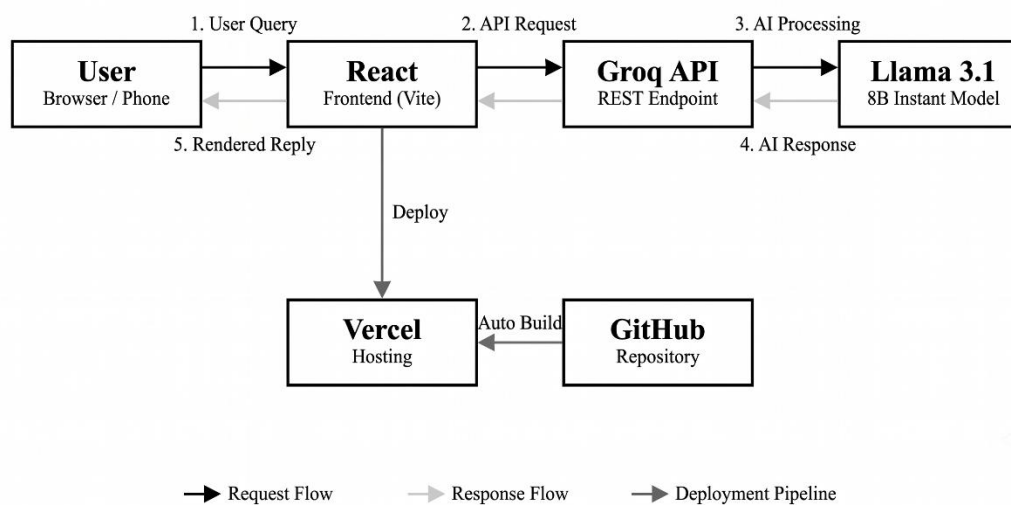


Figure 1: System Architecture of the Arbaeen Pilgrim Assistant Chatbot

3.2 Architecture Components

3.2.1 Frontend Layer

The frontend is a single-page application built with React.js using the Vite build tool. It consists of two primary source files. The Chat component manages the entire user interface including the message thread, the quick question buttons, the text input area, and the typing

indicator. The Groq service file handles all communication with the external API including request construction, authentication, and response extraction. The application uses React state hooks to manage conversation history, loading state, and user input without requiring any external state management library.

3.2.2 AI Layer

The AI layer consists of the Groq API endpoint which hosts the Llama 3.1 8B Instant model. Each user message triggers an API call that sends the full conversation history along with a fixed system prompt. The system prompt establishes the assistant persona, defines the three permitted topic areas, sets the tone of responses, and specifies emergency escalation behavior. The model generates a response which the frontend then renders in the chat interface.

3.2.3 Deployment Layer

The application is deployed on Vercel using its free tier. Vercel automatically builds the React application from the GitHub repository and serves it over HTTPS. Environment variables including the Groq API key are stored securely in Vercel and injected at build time, ensuring the key is never exposed in the source code or the public repository.

3.3 Data Flow

The data flow of a single user interaction follows four steps. First, the user types a question or selects a quick question in the browser. Second, the React frontend appends the message to the conversation history and sends a POST request to the Groq API, including the full history and the system prompt. Third, the Groq API processes the request using the Llama 3.1 8B model and returns a JSON response containing the assistant reply. Fourth, the frontend extracts the reply text and appends it to the conversation display.

3.4 Technology Stack Summary

Component	Technology
Frontend Framework	React.js with Vite
AI Model	Llama 3.1 8B Instant
AI API Provider	Groq (free tier)
HTTP Client	Axios
Hosting	Vercel (free tier)
Version Control	Git and GitHub

Live URL	https://arbaeen-chatbot.vercel.app
----------	---

4. Velocity Chart

4.1 Definition

Velocity measures the amount of work a team completes in a single sprint, expressed in story points. Tracking velocity across sprints allows the team to evaluate its pace, identify bottlenecks, and adjust sprint planning for future iterations. In this project, velocity is tracked across four sprints covering a total of 28 story points distributed across 11 user stories. Figure 2 presents the velocity data as a bar chart, illustrating the distribution of completed story points across all four sprints.

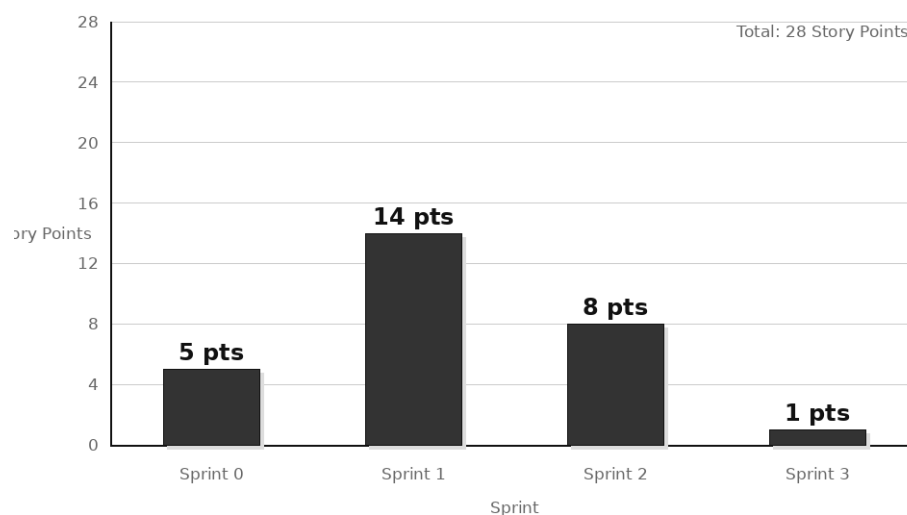


Figure 2: Sprint Velocity Chart showing story points completed per sprint

4.2 Velocity Data

Sprint	Planned Points	Completed Points	Completion Rate
Sprint 0	5	5	100%
Sprint 1	14	14	100%
Sprint 2	8	8	100%
Sprint 3	1	1	100%

Total	28	28	100%
--------------	-----------	-----------	-------------

4.3 Analysis

The team maintained a 100 percent completion rate across all four sprints. Sprint 1 carried the highest velocity at 14 story points, reflecting the concentration of core feature development in that phase. Sprint 0 and Sprint 3 carried lighter loads of 5 and 1 points respectively, which is consistent with their roles as setup and finalization sprints. The average velocity per sprint is 7 story points. The consistent completion of all planned work across every sprint demonstrates effective sprint planning and scope management throughout the project.

5. Lessons Learned

5.1 Project Management

Working within an agile framework for the first time on a real product taught me that sprint planning requires more discipline than it appears. The most important lesson I learned is that defining a realistic scope before a sprint begins is more valuable than trying to add features during development. In Sprint 1, where the workload was heaviest, I found that having clearly defined user stories with explicit acceptance criteria made it much easier to know when a feature was truly complete rather than merely functional.

5.2 AI-Assisted Development

Using AI tools throughout the development process significantly reduced the time required to produce working code and documentation. However, I learned that AI assistance works best when the developer has a clear mental model of what is needed before asking. Vague prompts produced vague outputs that required extensive rework. Specific and well-structured requests produced outputs that required only minor adjustments. This taught me that AI tools amplify the quality of engineering thinking rather than replacing it.

5.3 System Prompt Engineering

One of the most technically interesting challenges in this project was designing the system prompt that governs the chatbot behavior. I learned that the system prompt functions as the product specification for the AI layer. A poorly written prompt produces an assistant that

wanders off-topic or gives generic responses. A well-crafted prompt constrains the model to a specific domain while keeping its responses natural and helpful. The final prompt we developed went through several iterations before producing consistently relevant and accurate responses across all three use cases.

5.4 Deployment and Environment Management

Deploying a frontend application for the first time using Vercel and GitHub taught me the importance of environment variable management. Storing the API key in a local environment file and injecting it through Vercel at build time was a practical lesson in separating configuration from code. I also learned that testing locally using WSL on Windows requires careful attention to path and environment differences that do not exist on native Linux systems.

5.5 Scope Management

One of the most valuable decisions made during this project was consciously limiting the scope to three core features: Mawkib guidance, first aid advice, and route assistance. Early in the project there were ideas for multilingual support, GPS integration, and a missing persons feature. Keeping those out of the MVP scope allowed the team to deliver a complete and polished product within the available time. I now understand that a focused product that works well is far more valuable than an ambitious product that is half finished.

5.6 What I Would Do Differently

If I were to repeat this project, I would set up user testing earlier. Even informal testing with two or three people in the first week of development would have provided feedback that could have shaped the quick question choices and the system prompt design. I would also document the sprint board and burndown data in real time rather than reconstructing it retrospectively, since live documentation produces more accurate and credible agile artifacts.

6. Conclusion

This project successfully delivered a functional and deployed web-based chatbot designed to assist pilgrims during the Arbaeen walk. The system meets all three objectives defined in the project proposal: it provides Mawkib guidance, first aid advice, and route assistance through a simple and accessible conversational interface. The live product is available at arbaeen-chatbot.vercel.app and the complete source code is published on GitHub.

The project demonstrated that a meaningful and socially relevant software product can be built within a short academic timeline when agile methodology is applied consistently and AI development tools are used intelligently. The use of the Groq free tier with the Llama 3.1 8B model proved to be a reliable and cost-effective choice for an academic project of this scope. Looking ahead, several enhancements could extend the value of the system. A formal multilingual interface supporting Arabic and Persian would make the chatbot accessible to a wider range of pilgrims. Integration with a real-time Mawkib database would improve the accuracy of location guidance. Additionally, an offline mode would allow pilgrims to use the system in areas with limited internet connectivity, which is a common condition along parts of the walking route.

7. Project Links

The following links provide direct access to the deployed application and the complete source code repository:

7.1 Live Application

<https://arbaeen-chatbot.vercel.app>

7.2 GitHub Repository

<https://github.com/MohTayyar/arbaeen-chatbot>